

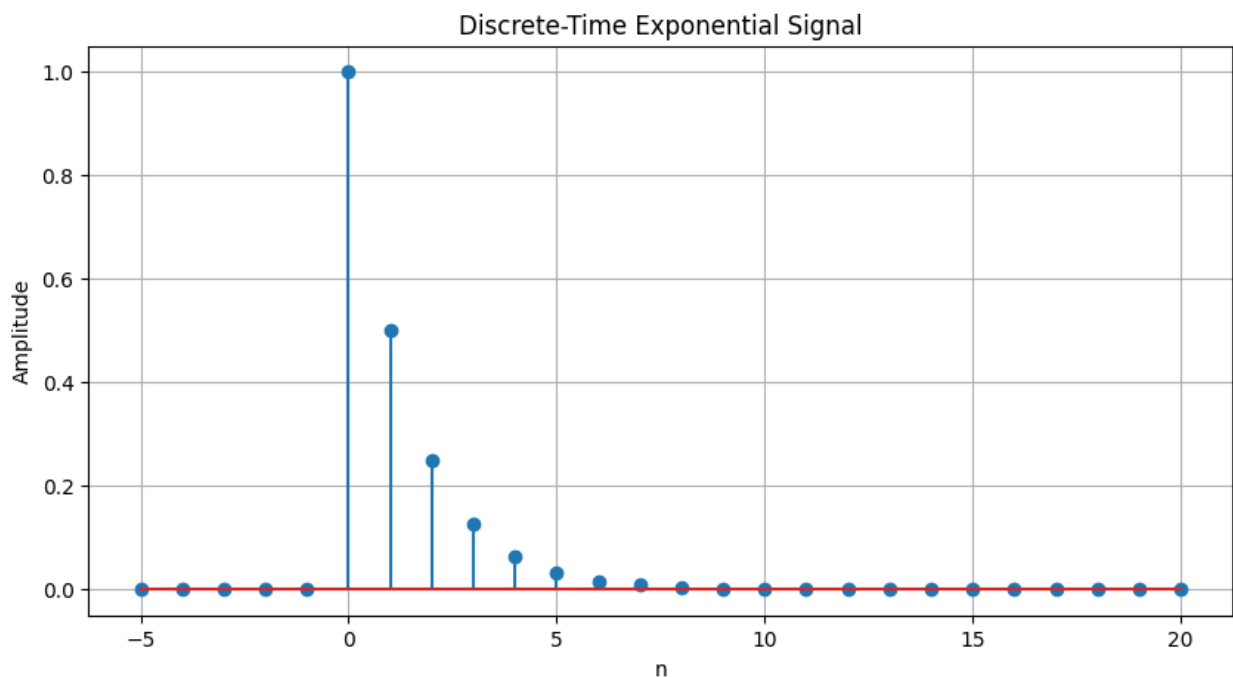
Module 5

Hints and Solutions

Section A — Basic Problems

Problem 1

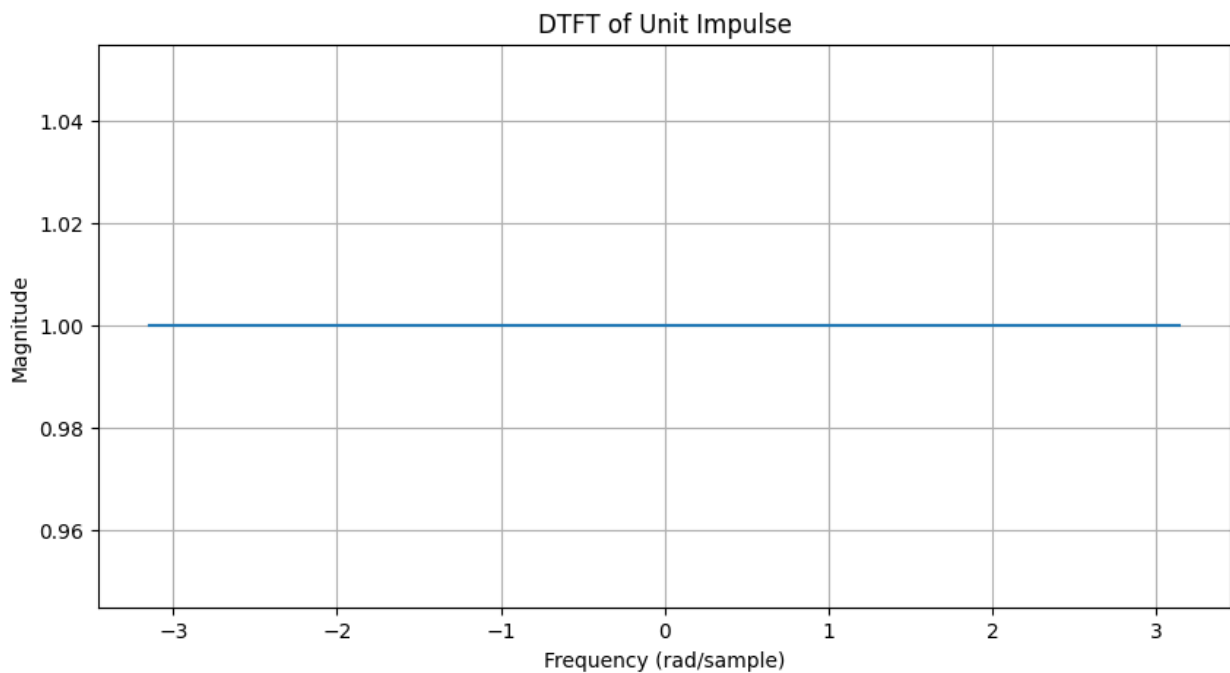
```
import numpy as np
import matplotlib.pyplot as plt
n = np.arange(-5,21) # Discrete-time index
u = np.where(n>=0,1,0) # Unit step
x = (0.5)**n * u # Signal
plt.figure(figsize=(10,5))
plt.stem(n,x)
plt.title("Discrete-Time Exponential Signal")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
print("Maximum Amplitude =", np.max(x))
print("Signal is exponentially decaying.")
```



```
Maximum Amplitude = 1.0
Signal is exponentially decaying.
```

Problem 2

```
import numpy as np
import matplotlib.pyplot as plt
N = 128
x = np.zeros(N)
x[0]=1
X = np.fft.fftshift(np.fft.fft(x))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X))
plt.title("DTFT of Unit Impulse")
plt.xlabel("Frequency (rad/sample)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```



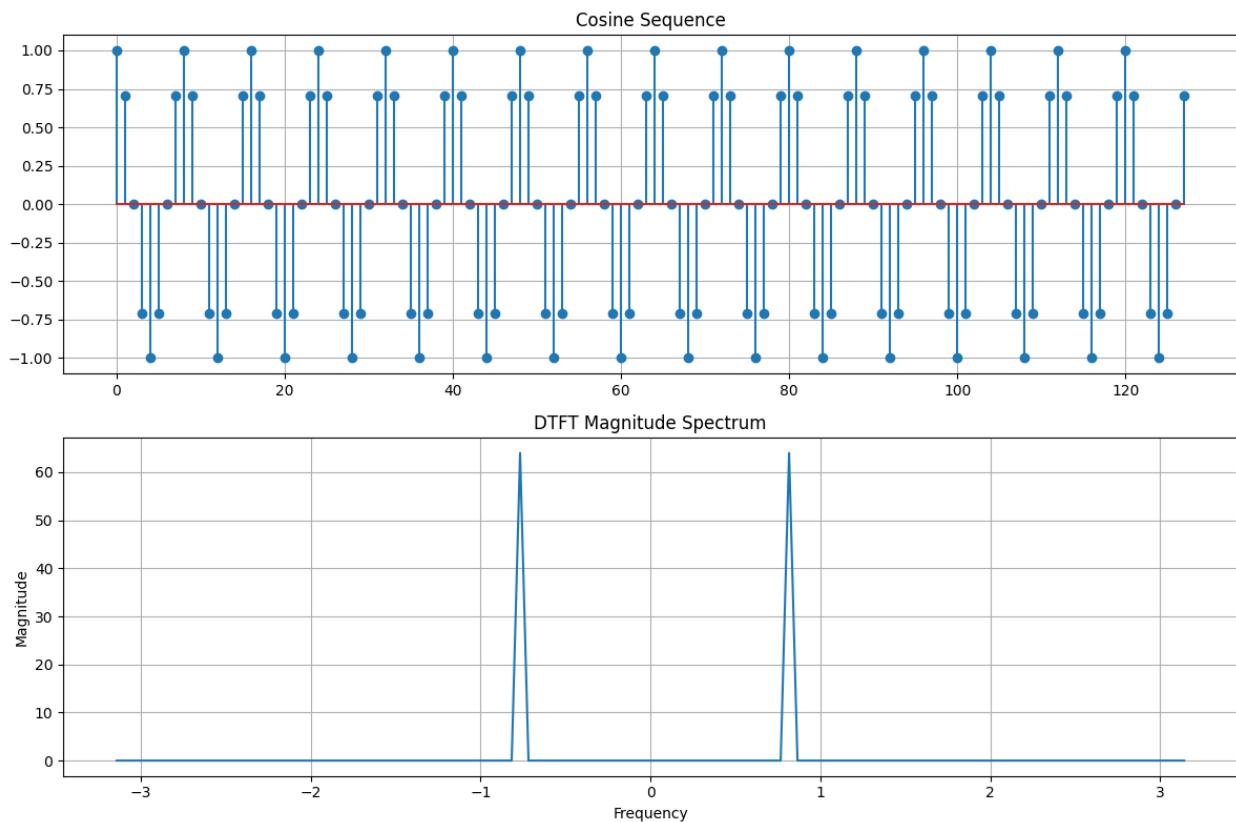
Problem 3

```
import numpy as np
import matplotlib.pyplot as plt
N = 128
n = np.arange(N)
x = np.cos(np.pi*n/4)
X = np.fft.fftshift(np.fft.fft(x))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
plt.stem(n,x)
```

```

plt.title("Cosine Sequence")
plt.grid(True)
plt.subplot(2,1,2)
plt.plot(w,np.abs(X))
plt.title("DTFT Magnitude Spectrum")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.tight_layout()
plt.show()

```



Problem 4

```

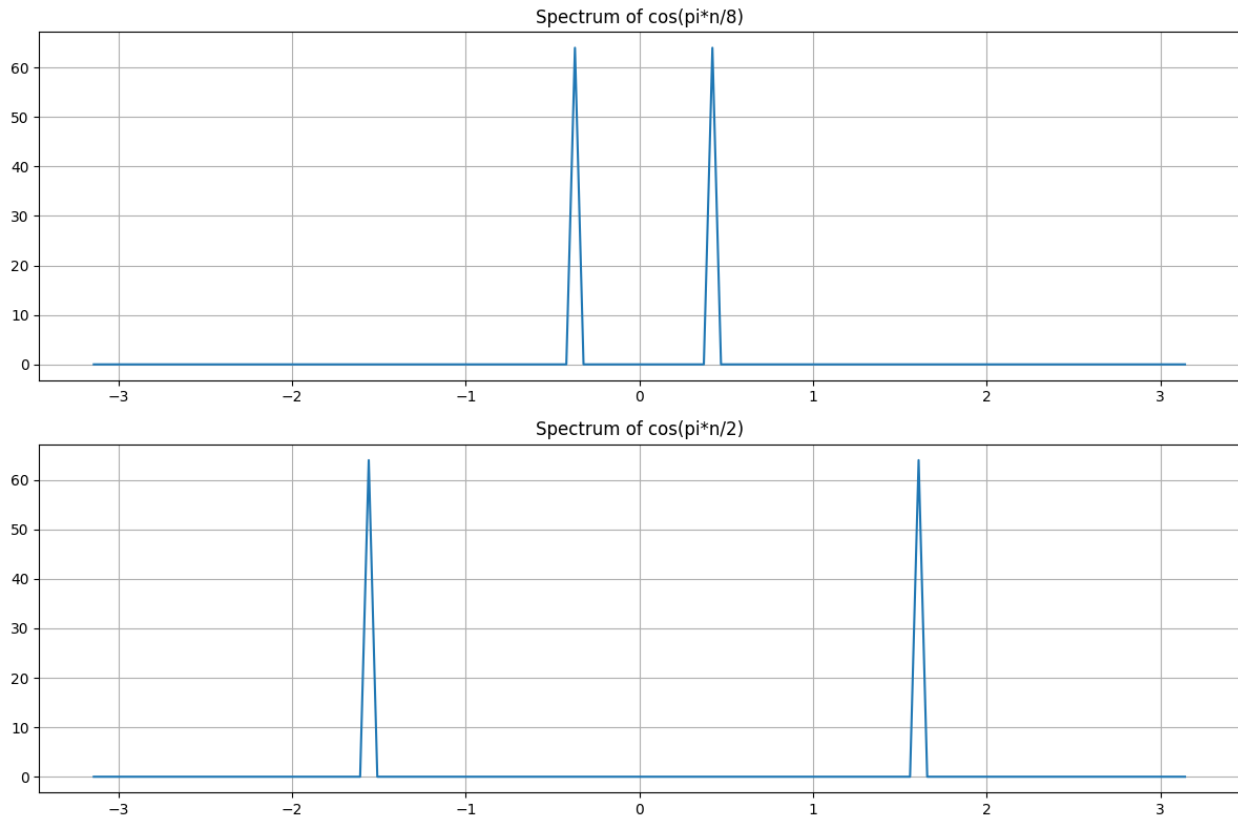
import numpy as np
import matplotlib.pyplot as plt
N = 128
n = np.arange(N)
x1 = np.cos(np.pi*n/8)
x2 = np.cos(np.pi*n/2)
X1 = np.fft.fftshift(np.fft.fft(x1))
X2 = np.fft.fftshift(np.fft.fft(x2))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)

```

```

plt.plot(w,np.abs(X1))
plt.title("Spectrum of  $\cos(\pi n/8)$ ")
plt.grid(True)
plt.subplot(2,1,2)
plt.plot(w,np.abs(X2))
plt.title("Spectrum of  $\cos(\pi n/2)$ ")
plt.grid(True)
plt.tight_layout()
plt.show()

```



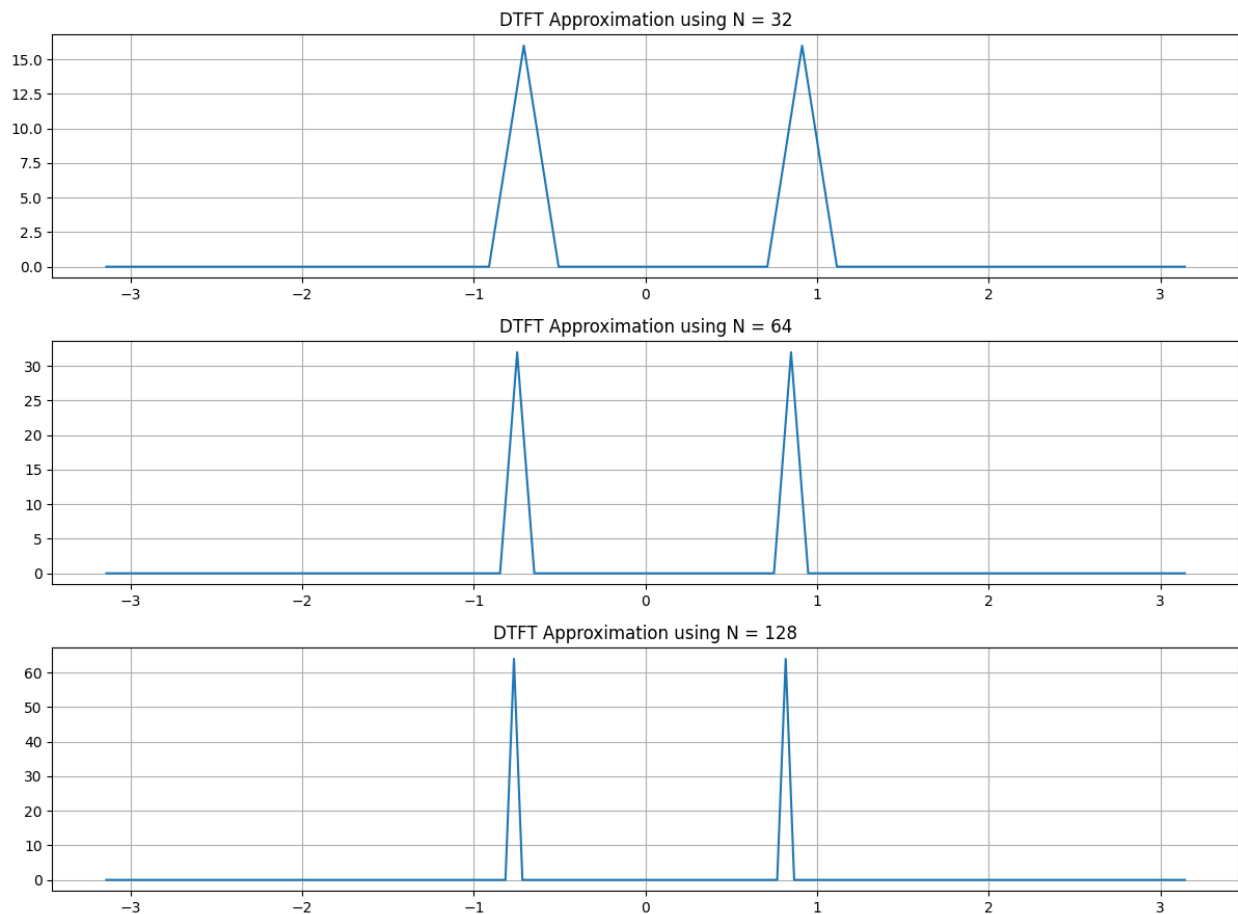
Problem 5

```

import numpy as np
import matplotlib.pyplot as plt
lengths = [32,64,128]
plt.figure(figsize=(12,9))
for i,N in enumerate(lengths):
    n = np.arange(N)
    x = np.cos(np.pi*n/4)
    X = np.fft.fftshift(np.fft.fft(x))
    w = np.linspace(-np.pi,np.pi,N)
    plt.subplot(3,1,i+1)
    plt.plot(w,np.abs(X))
    plt.title(f"DTFT Approximation using N = {N}")

```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```



Section B — DTFT Properties

These verification questions should be done both theoretically and with code for more conceptual clarity.

Problem 6

Hint

Generate two discrete-time signals.

Form a weighted sum of the signals.

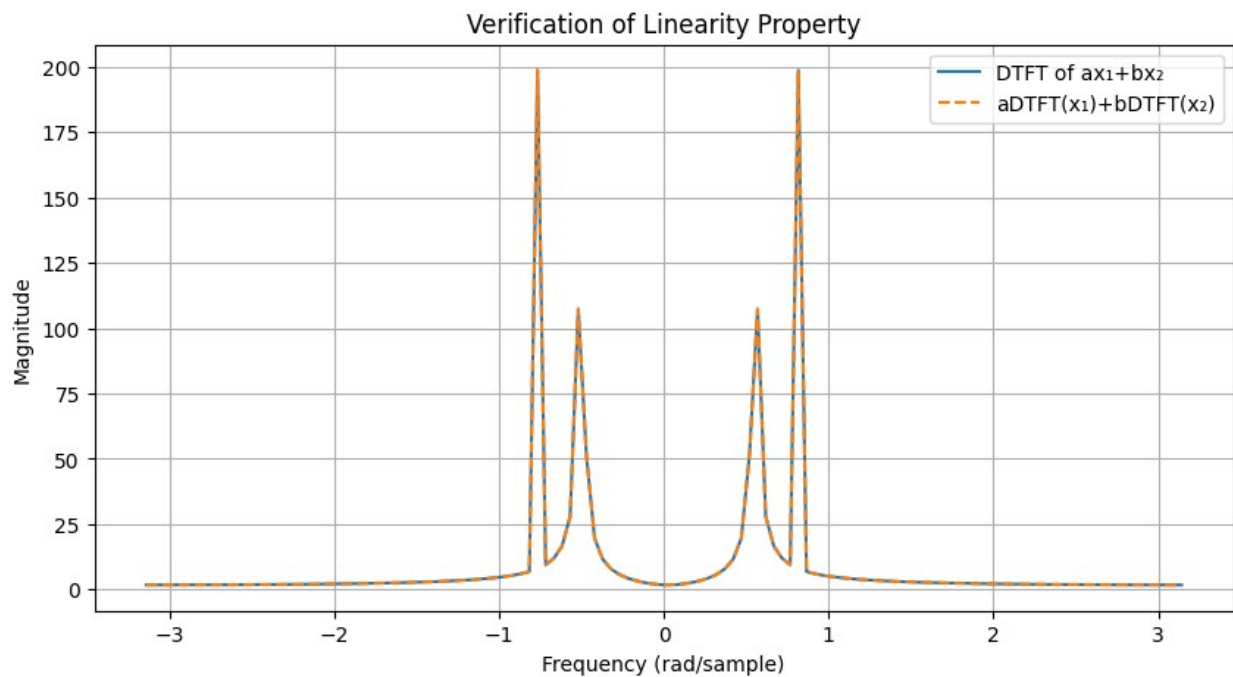
Compare the DTFT of the weighted sum with the weighted sum of the individual DTFTs.

```
import numpy as np
import matplotlib.pyplot as plt
```

```

N = 128
n = np.arange(N)
x1 = np.cos(np.pi*n/6)
x2 = np.sin(np.pi*n/4)
a = 2
b = 3
x = a*x1 + b*x2
X = np.fft.fftshift(np.fft.fft(x))
X1 = np.fft.fftshift(np.fft.fft(x1))
X2 = np.fft.fftshift(np.fft.fft(x2))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X),label="DTFT of  $ax_1+bx_2$ ")
plt.plot(w,np.abs(a*X1+b*X2),'--',label="aDTFT( $x_1$ )+bDTFT( $x_2$ )")
plt.legend()
plt.grid(True)
plt.title("Verification of Linearity Property")
plt.xlabel("Frequency (rad/sample)")
plt.ylabel("Magnitude")
plt.show()

```



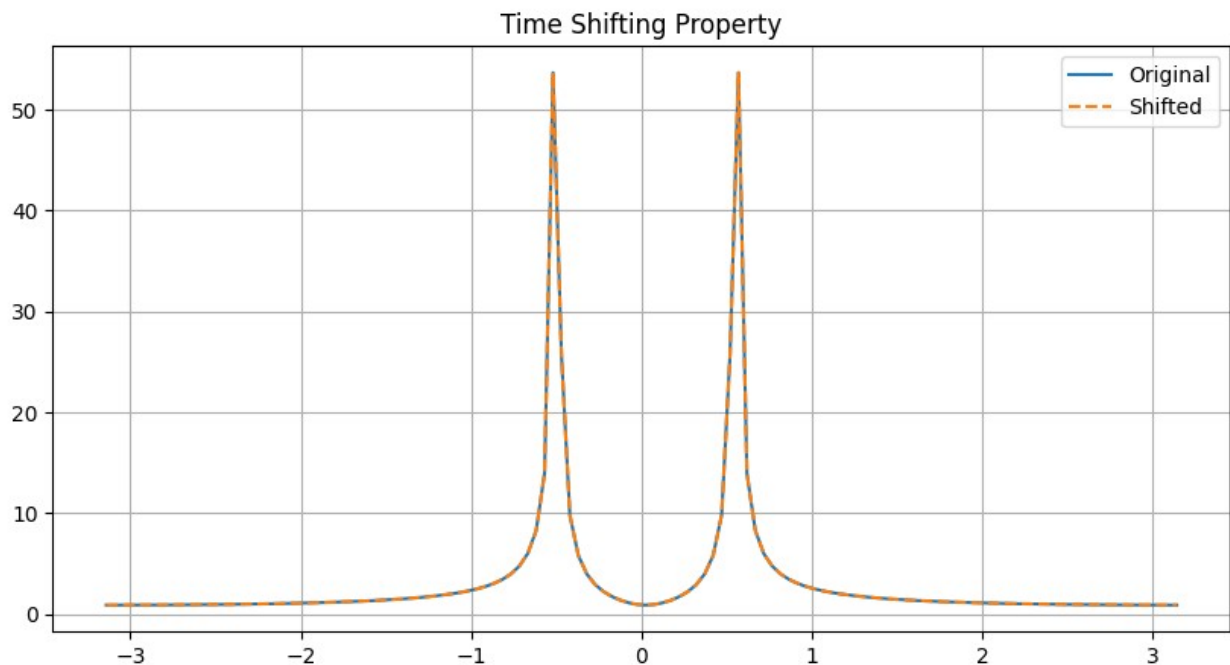
Problem 7

Hint

Delay a discrete-time sequence.

Compare the magnitude spectra.

```
import numpy as np
import matplotlib.pyplot as plt
N = 128
n = np.arange(N)
x = np.cos(np.pi*n/6)
delay = 10
x_shift = np.roll(x,delay)
X = np.fft.fftshift(np.fft.fft(x))
Xs = np.fft.fftshift(np.fft.fft(x_shift))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X),label="Original")
plt.plot(w,np.abs(Xs),'--',label="Shifted")
plt.legend()
plt.grid(True)
plt.title("Time Shifting Property")
plt.show()
```



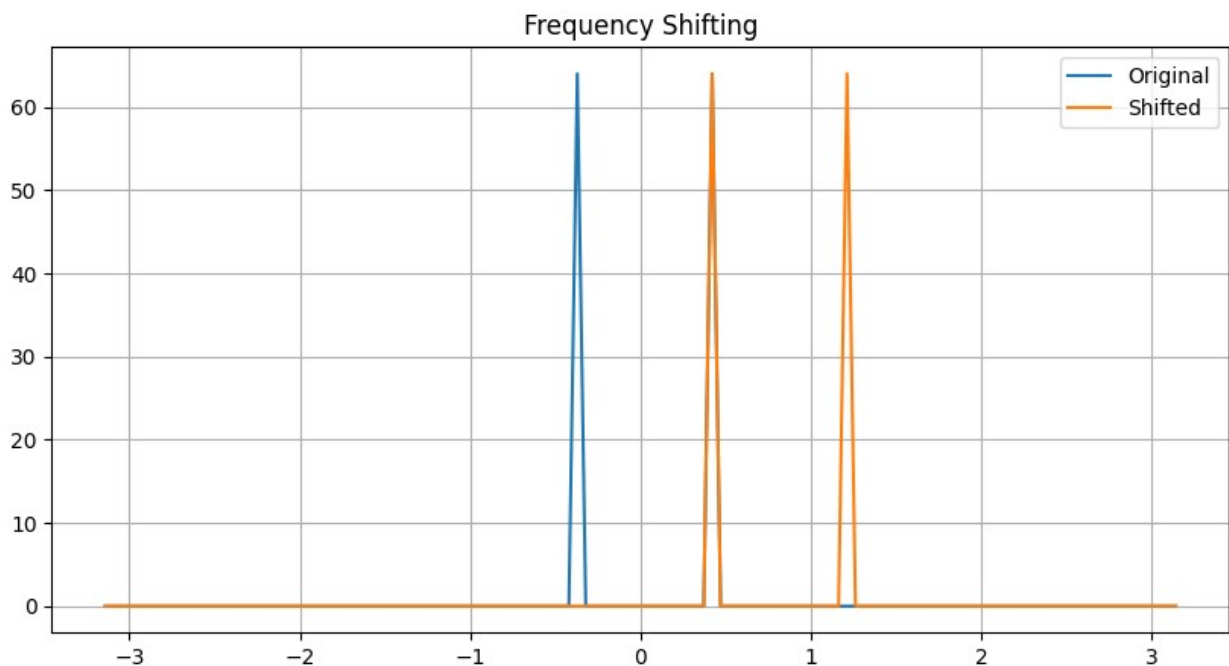
Problem 8

```
import numpy as np
import matplotlib.pyplot as plt
```

```

N = 128
n = np.arange(N)
x = np.cos(np.pi*n/8)
carrier = np.exp(1j*np.pi*n/4)
y = x*carrier
X = np.fft.fftshift(np.fft.fft(x))
Y = np.fft.fftshift(np.fft.fft(y))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X),label="Original")
plt.plot(w,np.abs(Y),label="Shifted")
plt.legend()
plt.grid()
plt.title("Frequency Shifting")
plt.show()

```



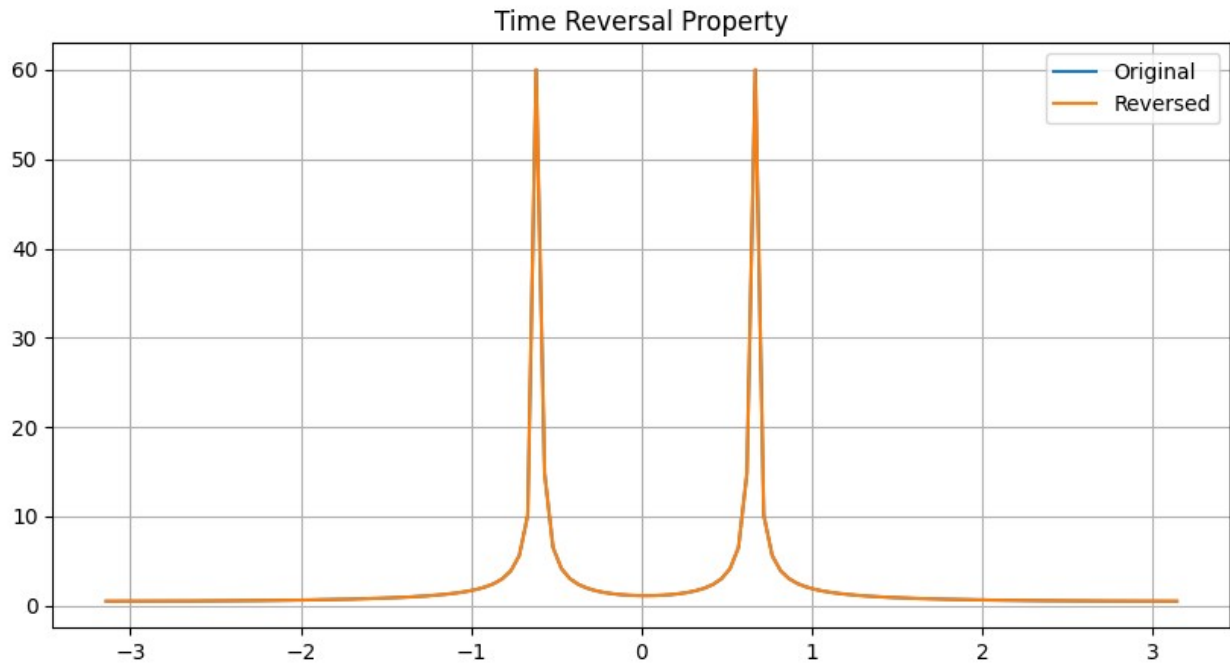
Problem 9

```

import numpy as np
import matplotlib.pyplot as plt
N = 128
n = np.arange(N)
x = np.cos(np.pi*n/5)
x_rev = x[::-1]
X = np.fft.fftshift(np.fft.fft(x))
XR = np.fft.fftshift(np.fft.fft(x_rev))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))

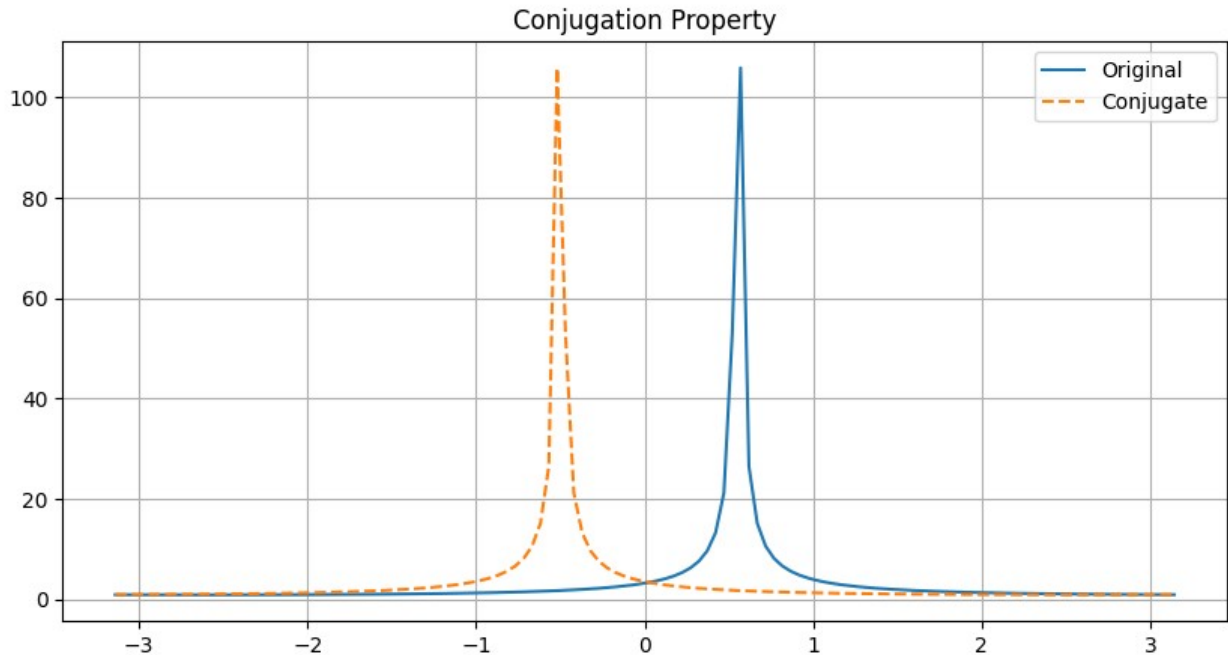
```

```
plt.plot(w,np.abs(X),label="Original")
plt.plot(w,np.abs(XR),label="Reversed")
plt.legend()
plt.grid()
plt.title("Time Reversal Property")
plt.show()
```



Problem 10

```
import numpy as np
import matplotlib.pyplot as plt
N = 128
n = np.arange(N)
x = np.exp(1j*np.pi*n/6)
x_conj = np.conjugate(x)
X = np.fft.fftshift(np.fft.fft(x))
XC = np.fft.fftshift(np.fft.fft(x_conj))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X),label="Original")
plt.plot(w,np.abs(XC),'--',label="Conjugate")
plt.legend()
plt.grid()
plt.title("Conjugation Property")
plt.show()
```



Section C — Programming Problems

Problem 11

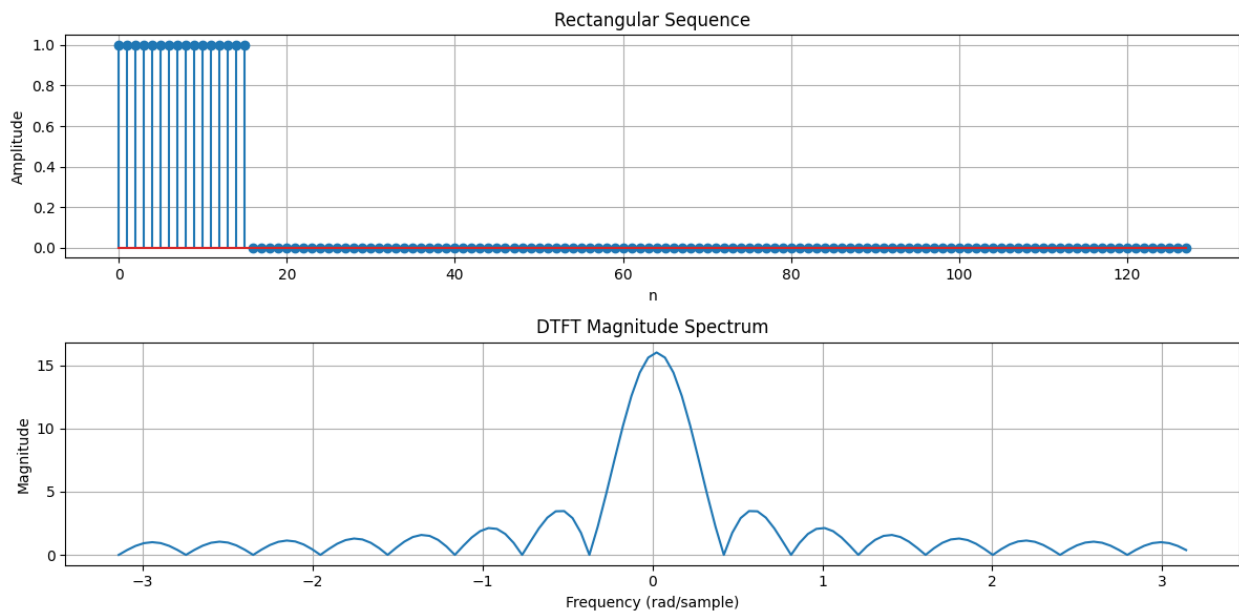
```
import numpy as np
import matplotlib.pyplot as plt
N = 128
L = 16
x = np.zeros(N)
x[:L] = 1
X = np.fft.fftshift(np.fft.fft(x))
w = np.linspace(-np.pi, np.pi, N)

plt.figure(figsize=(12,6))

plt.subplot(2,1,1)
plt.stem(np.arange(N), x)
plt.title("Rectangular Sequence")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(w, np.abs(X))
plt.title("DTFT Magnitude Spectrum")
plt.xlabel("Frequency (rad/sample)")
plt.ylabel("Magnitude")
plt.grid(True)
```

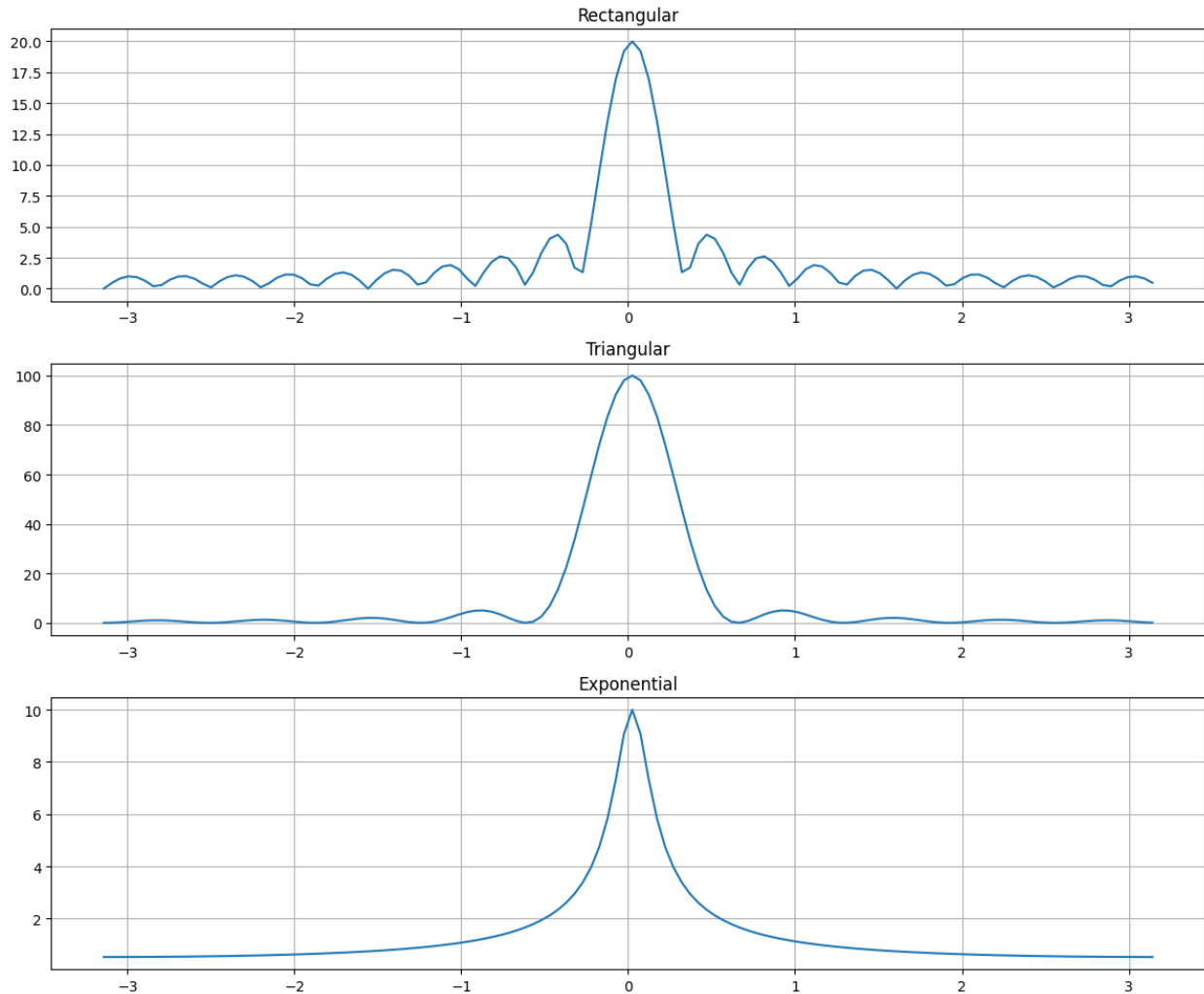
```
plt.tight_layout()
plt.show()
```



Problem 12

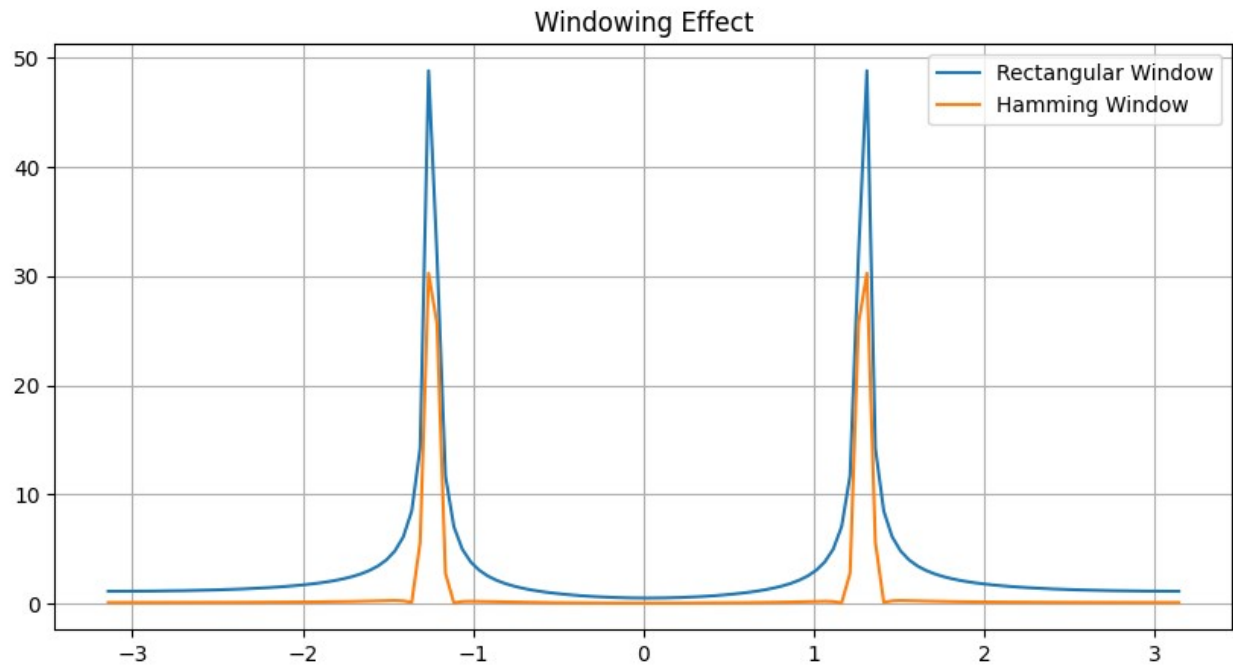
```
N = 128
n = np.arange(N)
rect = np.zeros(N)
rect[:20] = 1
tri = np.concatenate((np.arange(1,11), np.arange(9,0,-1)))
tri = np.pad(tri,(0,N-len(tri)))
expo = (0.9)**n
signals = [rect, tri, expo]
titles = ["Rectangular","Triangular","Exponential"]
plt.figure(figsize=(12,10))
for i,s in enumerate(signals):
    X = np.fft.fftshift(np.fft.fft(s))
    plt.subplot(3,1,i+1)
    plt.plot(w,np.abs(X))
    plt.title(titles[i])
    plt.grid(True)

plt.tight_layout()
plt.show()
```



Problem 13

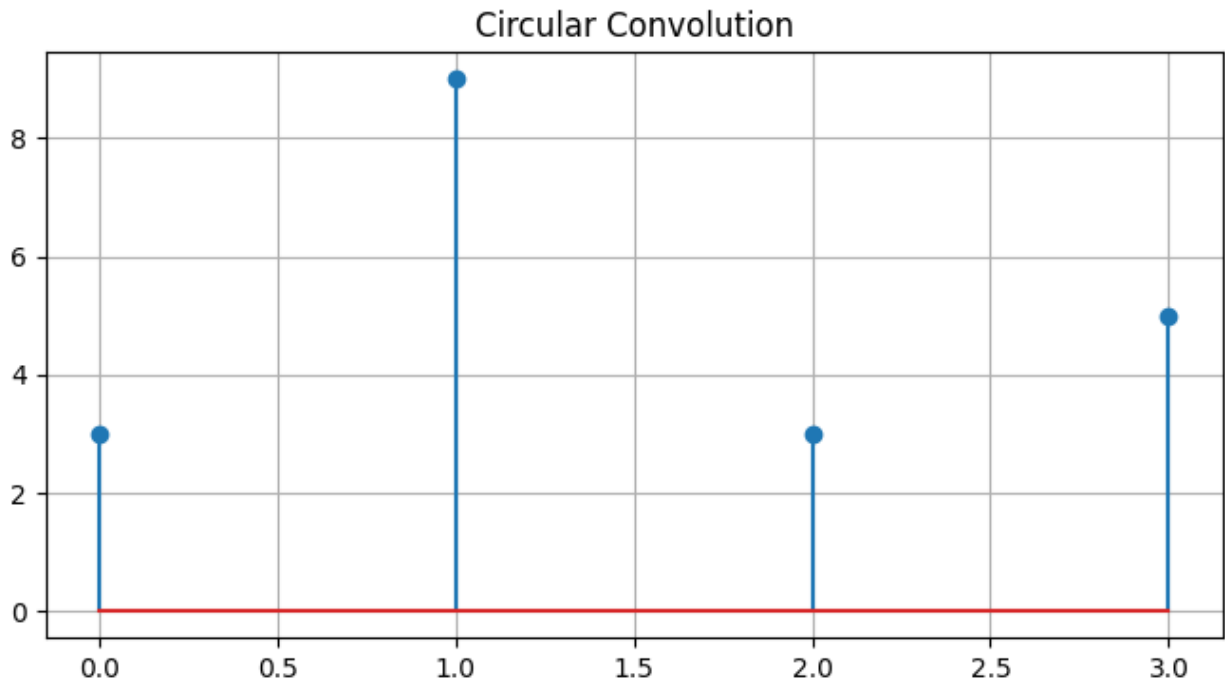
```
N = 128
n = np.arange(N)
signal = np.cos(2*np.pi*0.2*n)
rect_window = np.ones(N)
hamming_window = np.hamming(N)
x1 = signal * rect_window
x2 = signal * hamming_window
X1 = np.fft.fftshift(np.fft.fft(x1))
X2 = np.fft.fftshift(np.fft.fft(x2))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X1),label="Rectangular Window")
plt.plot(w,np.abs(X2),label="Hamming Window")
plt.legend()
plt.grid(True)
plt.title("Windowing Effect")
plt.show()
```



Problem 14

```
x = np.array([1,2,3,4])
h = np.array([1,-1,2,0])
X = np.fft.fft(x)
H = np.fft.fft(h)
Y = X * H
y = np.real(np.fft.ifft(Y))
print("Circular Convolution Result")
print(y)
plt.figure(figsize=(8,4))
plt.stem(y)
plt.title("Circular Convolution")
plt.grid(True)
plt.show()
```

Circular Convolution Result
[3. 9. 3. 5.]

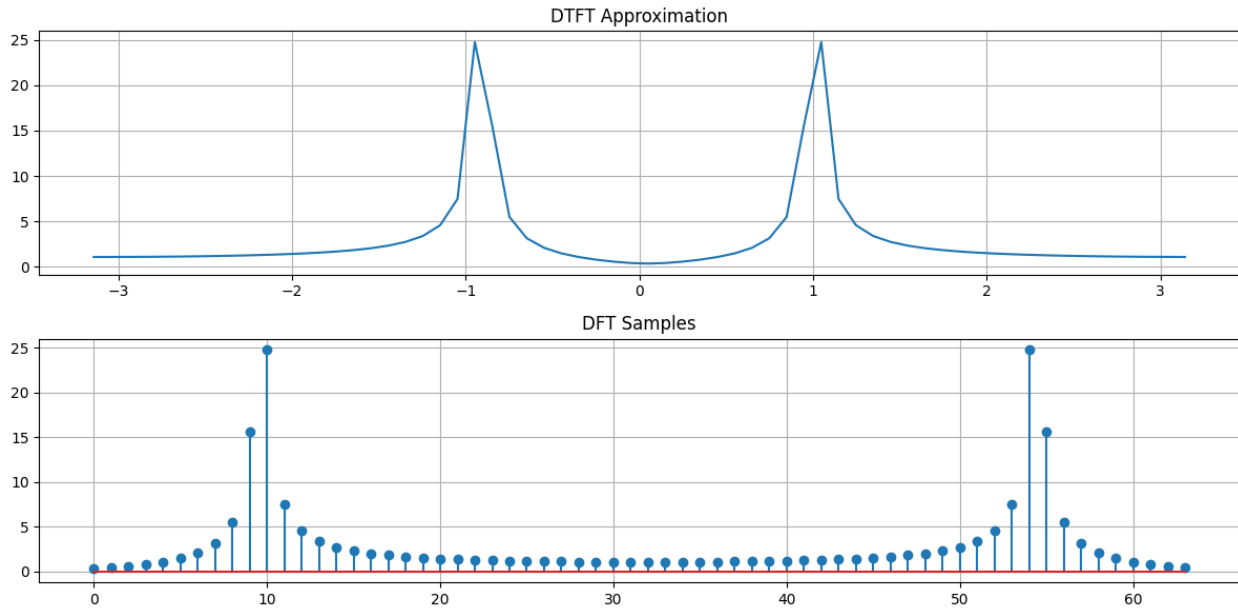


Problem 15

```

N = 64
n = np.arange(N)
x = np.cos(2*np.pi*0.15*n)
X = np.fft.fftshift(np.fft.fft(x))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(12,6))
plt.subplot(2,1,1)
plt.plot(w,np.abs(X))
plt.title("DTFT Approximation")
plt.grid(True)
plt.subplot(2,1,2)
plt.stem(np.arange(N),np.abs(np.fft.fft(x)))
plt.title("DFT Samples")
plt.grid(True)
plt.tight_layout()
plt.show()

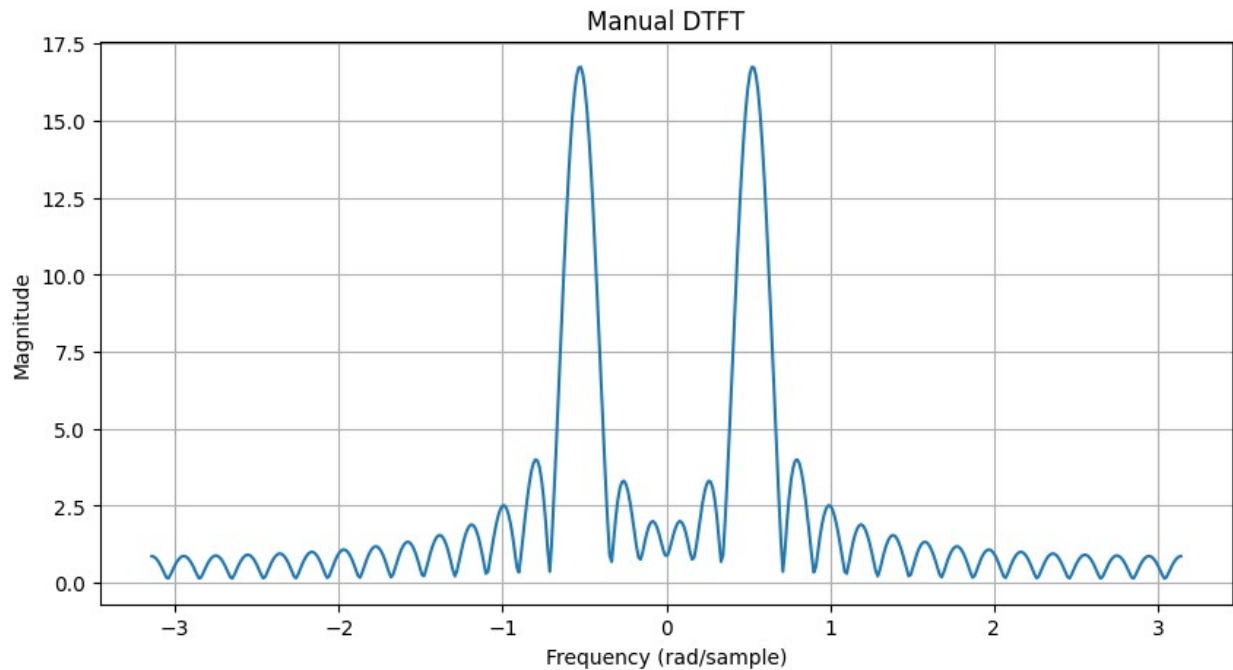
```



Challenge Problems

Challenge Problem 1

```
import numpy as np
import matplotlib.pyplot as plt
N = 32
n = np.arange(N)
x = np.cos(np.pi*n/6)
w = np.linspace(-np.pi,np.pi,500)
X = np.zeros(len(w),dtype=complex)
for i,omega in enumerate(w):
    X[i] = np.sum(x*np.exp(-1j*omega*n))
plt.figure(figsize=(10,5))
plt.plot(w,np.abs(X))
plt.title("Manual DTFT")
plt.xlabel("Frequency (rad/sample)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

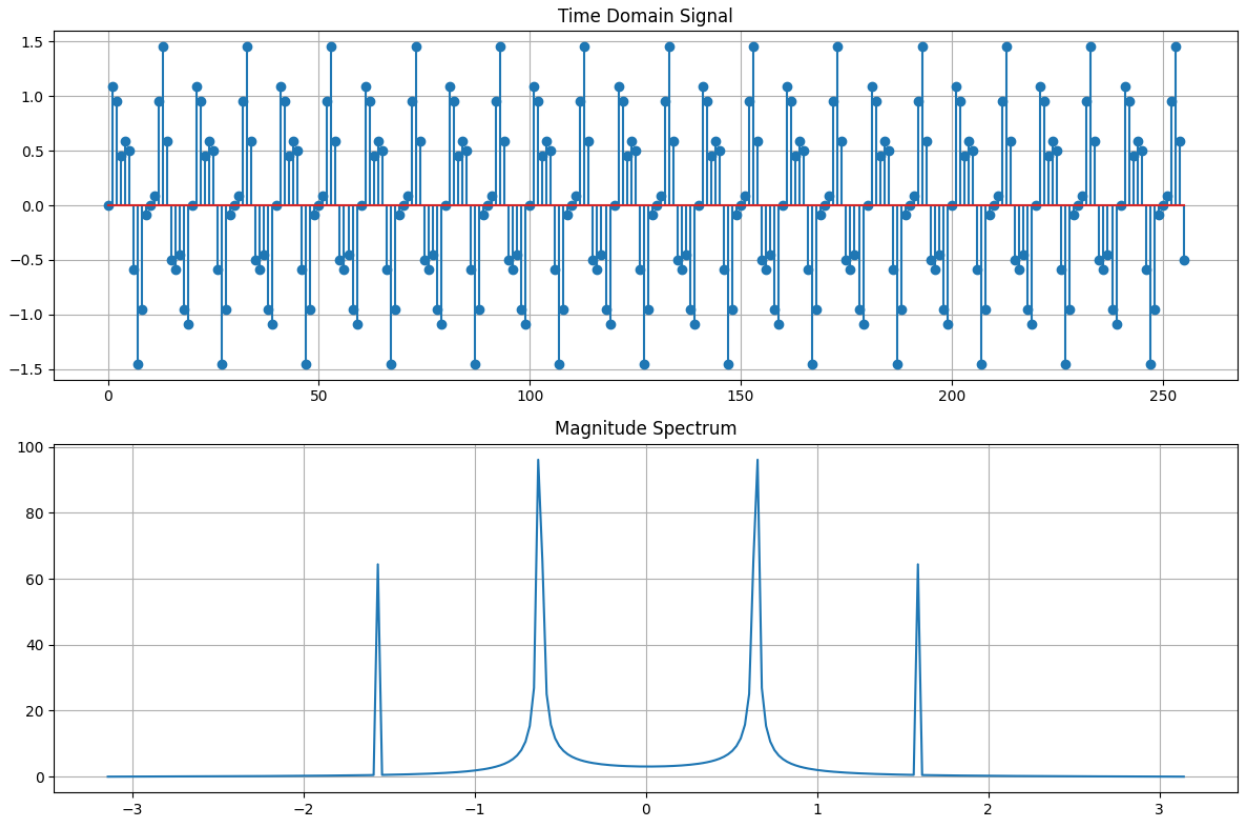


Challenge Problem 2

```

N = 256
n = np.arange(N)
x = np.sin(0.2*np.pi*n)+0.5*np.sin(0.5*np.pi*n)
X = np.fft.fftshift(np.fft.fft(x))
w = np.linspace(-np.pi,np.pi,N)
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
plt.stem(n,x)
plt.title("Time Domain Signal")
plt.grid(True)
plt.subplot(2,1,2)
plt.plot(w,np.abs(X))
plt.title("Magnitude Spectrum")
plt.grid(True)
plt.tight_layout()
plt.show()

```



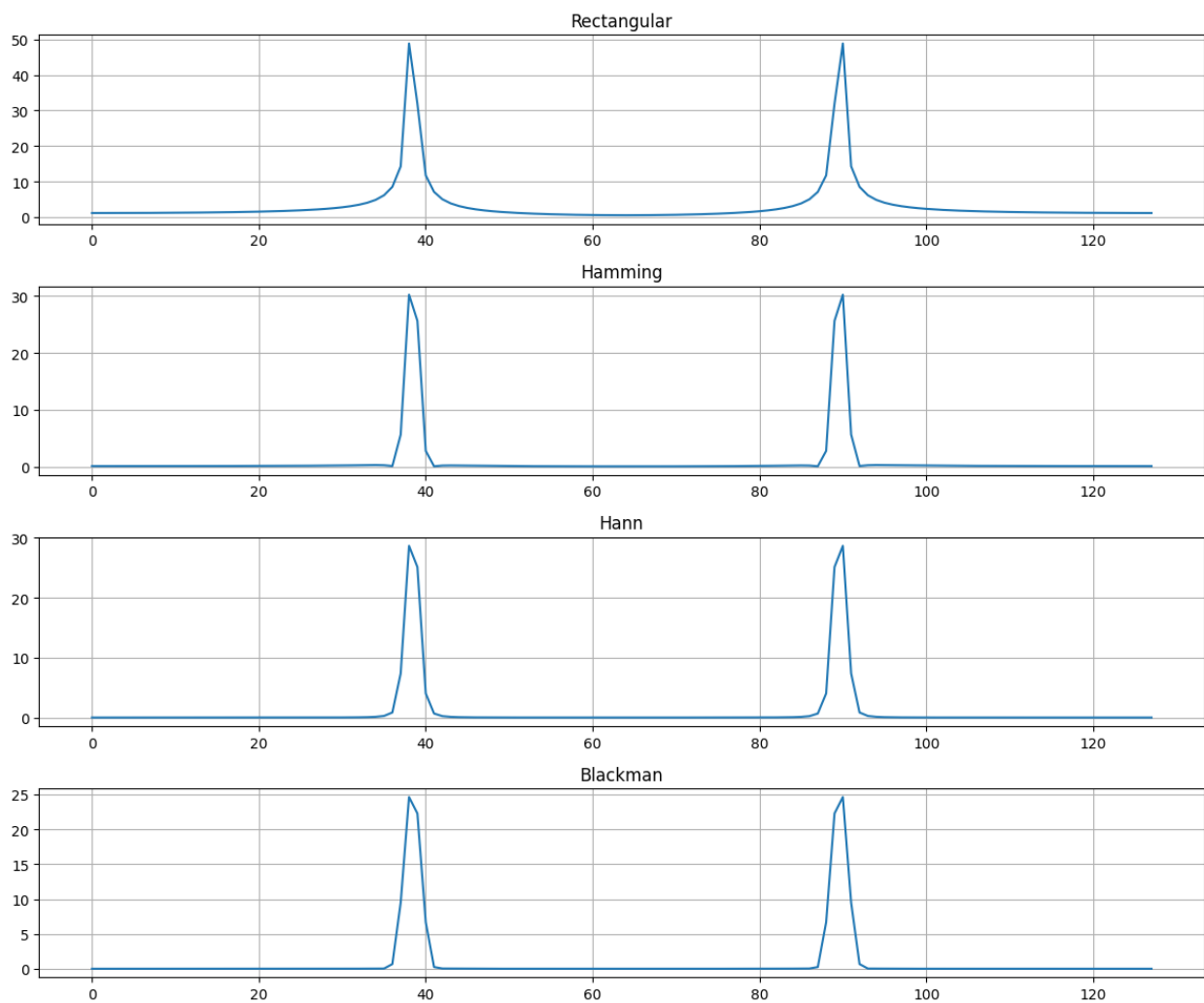
Challenge Problem 3

```

N = 128
n = np.arange(N)
signal = np.cos(2*np.pi*0.2*n)
windows = [
    np.ones(N),
    np.hamming(N),
    np.hanning(N),
    np.blackman(N)
]
titles = [
    "Rectangular",
    "Hamming",
    "Hann",
    "Blackman"
]
plt.figure(figsize=(12,10))
for i,w in enumerate(windows):
    x = signal*w
    X = np.fft.fftshift(np.fft.fft(x))
    plt.subplot(4,1,i+1)
    plt.plot(np.abs(X))
    plt.title(titles[i])
    plt.grid(True)

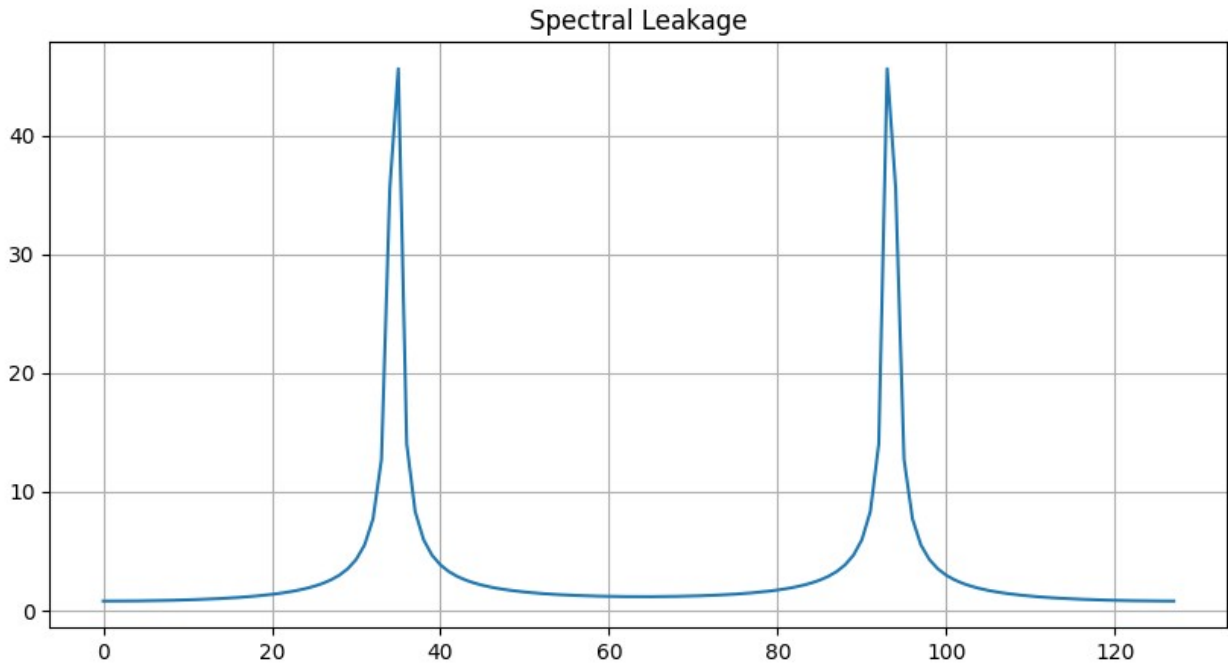
```

```
plt.tight_layout()
plt.show()
```



Challenge Problem 4

```
N = 128
n = np.arange(N)
x = np.cos(2*np.pi*0.23*n)
X = np.fft.fftshift(np.fft.fft(x))
plt.figure(figsize=(10,5))
plt.plot(np.abs(X))
plt.title("Spectral Leakage")
plt.grid(True)
plt.show()
```

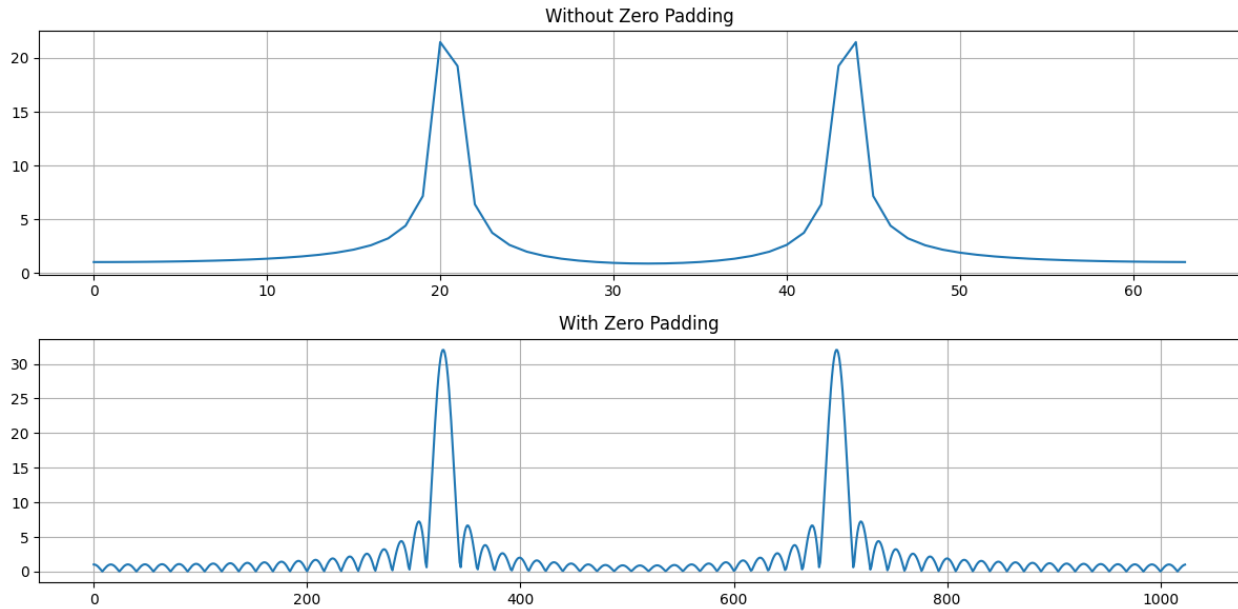


Challenge Problem 5

```

N = 64
n = np.arange(N)
x = np.cos(2*np.pi*0.18*n)
X1 = np.fft.fftshift(np.fft.fft(x))
X2 = np.fft.fftshift(np.fft.fft(x,1024))
plt.figure(figsize=(12,6))
plt.subplot(2,1,1)
plt.plot(np.abs(X1))
plt.title("Without Zero Padding")
plt.grid(True)
plt.subplot(2,1,2)
plt.plot(np.abs(X2))
plt.title("With Zero Padding")
plt.grid(True)
plt.tight_layout()
plt.show()

```



Challenge Problem 6

```
fs = 1000
t = np.arange(0,1,1/fs)
x = np.sin(2*np.pi*120*t)
X = np.fft.fft(x)
freq = np.fft.fftfreq(len(t),1/fs)
index = np.argmax(np.abs(X[:len(X)//2]))
print("Estimated Frequency =",freq[index],"Hz")
plt.figure(figsize=(10,5))
plt.plot(freq[:len(freq)//2],np.abs(X[:len(X)//2]))
plt.title("Frequency Detector")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```

Estimated Frequency = 120.0 Hz

